

AUTOMATIC PARKING SLOT DETECTION USING DIGITAL IMAGE PROCESSING

Name – G Harshaditya Reddy

Reg No – 23BCE0762

1. Introduction

With the increasing number of vehicles worldwide, efficient parking management has become an essential requirement in urban areas. Drivers often have to spend several minutes searching for available parking spaces, especially in crowded locations such as shopping malls, airports, office buildings, and public parking areas.

Traditional parking systems rely on manual monitoring or physical sensors installed in parking spaces which have several disadvantages such as high installation costs, maintenance issues, and limited scalability. As a result, researchers and engineers are exploring alternative approaches for parking management.

Digital Image Processing (DIP) provides an effective solution for automated parking detection. By analysing images captured by cameras installed in parking areas, it is possible to determine whether a parking space is occupied or empty. Image processing techniques allow computers to analyse visual information and extract meaningful features from images.

In this project, a parking slot detection system is developed using image processing techniques. The system processes parking lot images to detect vehicles and determine parking space occupancy. The system uses algorithms such as grayscale conversion, Gaussian

filtering, Canny edge detection, and contour detection to identify vehicles in the image.

The proposed solution provides a cost-effective and scalable method for parking management. By integrating such systems with real-time monitoring and mobile applications, drivers can be informed about available parking spaces before entering a parking area.

2. Literature Survey

Several researchers have investigated the use of image processing and computer vision techniques for parking management systems.

One study proposed the use of edge detection algorithms to identify vehicles in parking spaces. The method involves detecting object boundaries and identifying shapes corresponding to vehicles.

Although effective in detecting vehicle edges, the method can be sensitive to lighting conditions.

Another study explored the use of background subtraction techniques for vehicle detection. In this method, a reference image of an empty parking lot is compared with the current image to detect changes. These changes indicate the presence of vehicles in parking slots. While this technique is effective, it requires a stable background reference image.

Although these approaches provide higher accuracy, they require large datasets and significant computational resources. Compared to these advanced methods, traditional image processing techniques such as edge detection and contour detection are simpler to implement and require fewer computational resources. These techniques can effectively detect vehicles in parking spaces with reasonable accuracy.

Therefore, this project focuses on implementing a parking detection system using classical digital image processing techniques.

3. Algorithms Used

The system uses several fundamental digital image processing techniques to analyse parking lot images.

3.1 Grayscale Conversion

Colour images contain three channels: red, green, and blue. Processing all three channels increases computational complexity. Therefore, the input image is first converted into grayscale format. Grayscale images contain only intensity information, making it easier to apply image processing algorithms.

3.2 Gaussian Filtering

Images captured by cameras often contain noise caused by lighting variations and camera sensors. Gaussian filtering is used to smooth the image and reduce noise.

The Gaussian filter applies a weighted average to neighbouring pixels, resulting in a smoother image that is better suited for further processing.

3.3 Edge Detection

Edge detection is a technique used to identify boundaries of objects in an image. In this project, the Canny edge detection algorithm is used.

The Canny edge detection algorithm performs the following steps:

1. Noise reduction using Gaussian filtering

2. Gradient calculation
3. Non-maximum suppression
4. Edge tracking using hysteresis

This process identifies edges corresponding to vehicle boundaries.

3.4 Contour Detection

Contours are curves that connect continuous points along object boundaries. After edge detection, contour detection is applied to identify distinct shapes in the image.

Contours with large areas typically correspond to vehicles. By analysing contour size and shape, the system can detect the presence of cars in the parking lot.

4. System Architecture

The parking detection system follows a sequence of image processing steps to analyse parking lot images.

The workflow of the system is as follows:

1. Image Acquisition:
A parking lot image is captured using a camera or obtained from a dataset.
2. Image Preprocessing:
The image is converted to grayscale and smoothed using Gaussian filtering.
3. Edge Detection:
The Canny edge detection algorithm identifies object boundaries.

4. Contour Detection:

Contours are extracted from the edge image.

5. Vehicle Detection:

Contours with significant area are considered as vehicles.

6. Output Visualization:

Bounding boxes are drawn around detected vehicles to indicate occupied parking spaces.

This architecture allows the system to detect vehicles in parking spaces effectively.

5. Dataset Description

The dataset used in this project consists of parking lot images obtained from publicly available sources.

Dataset Name: Parking Lot Dataset

Source: Kaggle

Dataset Link:

<https://www.kaggle.com/datasets/blanderbuss/parking-lot-dataset>

The dataset contains images captured from overhead parking cameras. These images include multiple vehicles parked in designated parking spaces. The dataset provides a variety of scenarios, including different lighting conditions and parking arrangements.

The images are used as input to the system for testing the vehicle detection algorithm.

6. Hardware Requirements

The project does not require specialized hardware and can run on a standard personal computer.

Minimum hardware requirements include:

Processor: Intel Core i5 or higher

RAM: 16 GB or higher

Storage: 500 MB free disk space

Display: Standard monitor

Camera (optional): CCTV or parking lot camera for real-time implementation

7. Software Requirements

The software environment used for this project includes:

Programming Language: Python

Libraries used:

OpenCV – used for image processing operations such as filtering, edge detection, and contour detection.

NumPy – used for numerical operations and matrix manipulation.

Matplotlib – used for displaying images and visualization.

Development Environment:

Visual Studio Code or Jupyter Notebook

Operating System:

Windows, Linux, or macOS

8. Implementation

The implementation of the parking detection system is carried out using Python and the OpenCV library.

The main steps in the implementation are:

1. Load the parking lot image using OpenCV.
2. Convert the image to grayscale format.
3. Apply Gaussian blur to remove noise.
4. Perform edge detection using the Canny algorithm.
5. Detect contours from the edge image.
6. Filter contours based on area to identify vehicles.
7. Draw bounding boxes around detected vehicles.

The resulting image displays rectangles around vehicles detected in the parking lot.

9. Experimental Analysis

The system was tested using parking lot images containing multiple vehicles.

The results show that the algorithm successfully detects vehicles based on contour size and edge detection. Vehicles present in the parking lot are highlighted using bounding boxes.

However, the accuracy of detection depends on several factors such as lighting conditions, camera angle, and image resolution. In cases where vehicles overlap or where shadows are present, the algorithm may produce incorrect detections.

Despite these limitations, the system demonstrates that digital image processing techniques can effectively identify vehicles in parking areas.

10. Code

```
import cv2
```

```
import numpy as np
```

```
# Read parking image
```

```
img = cv2.imread('parking.jpg')
```

```
# Convert to grayscale
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Apply Gaussian Blur to remove noise
```

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

```
# Edge detection using Canny algorithm
```

```
edges = cv2.Canny(blur, 50, 150)
```

```
# Find contours from edge image
```



```
contours, _ = cv2.findContours(edges, cv2.RETR_TREE,  
cv2.CHAIN_APPROX_SIMPLE)
```

```
# Copy original image for output
```

```
output = img.copy()
```

```
# Loop through contours
```

```
for c in contours:
```

```
    area = cv2.contourArea(c)
```

```
    # Detect large contours (likely vehicles)
```

```
    if area > 2000:
```

```
        x, y, w, h = cv2.boundingRect(c)
```

```
        # Draw rectangle around detected cars
```

```
        cv2.rectangle(output, (x,y), (x+w, y+h), (0,255,0), 2)
```

```
# Show results
```

```
cv2.imshow("Original Image", img)
cv2.imshow("Edge Detection", edges)
cv2.imshow("Detected Vehicles", output)

# Wait until key press
cv2.waitKey(0)

# Close windows
cv2.destroyAllWindows()
```

11. Results

The output images generated by the system show detected vehicles highlighted with rectangular boxes.

The system successfully identifies large objects corresponding to parked vehicles. The detection accuracy is sufficient for demonstration purposes and basic parking monitoring systems.

The results confirm that contour detection combined with edge detection can effectively detect vehicles in parking lot images.

12. Conclusion

This project presents an automated parking slot detection system using digital image processing techniques. The system analyses

parking lot images to detect vehicles and identify occupied parking spaces.

The implementation uses simple but effective image processing algorithms such as grayscale conversion, Gaussian filtering, edge detection, and contour detection.

The results demonstrate that these techniques can successfully detect vehicles in parking areas with reasonable accuracy. The proposed system can be used as a foundation for developing smart parking systems in urban environments.

13. External Links

i) <https://arxiv.org/abs/2203.06463>

ii) <https://arxiv.org/abs/2005.05528>